

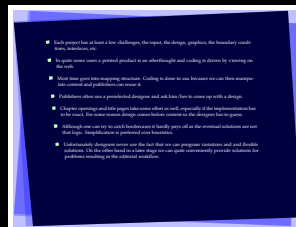
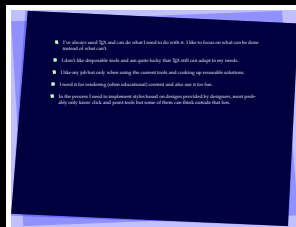
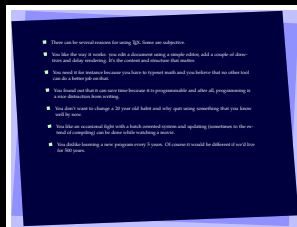
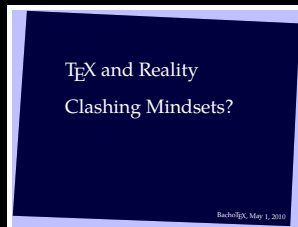
Presentations

This is a selection of three sets of presentations at meetings and a bunch of examples for some styles that come with **ConT_EXt**. In **MkIV** some of the older styles have been dropped. There are more presentations, starting mid 1990's: some are lost, others make no sense showing (or have articles instead) and I have no time anyway to collect all of them.

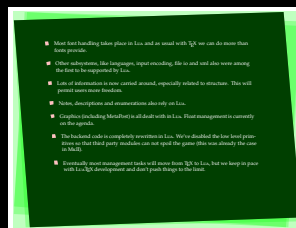
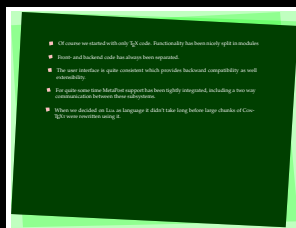
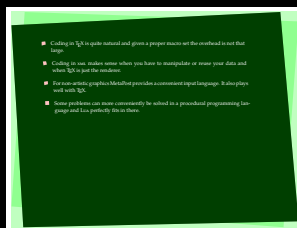
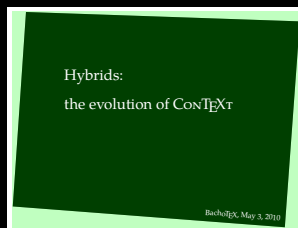
We show the first four pages but you can find the complete files in the **ConT_EXt** distribution. This file is generated automatically so it's also a snapshot. Maybe some day I find the time to add some more.

There are more files in the source tree because it doesn't make sense to provide all **pdf** files. Some files are to be generated with **LuaT_EX** because they discuss something (**MkIV**) that in the meantime has been superseded. Keep in mind that these talk are snapshots and might have presenting the state at that time, some experiments, or just have been teasers.

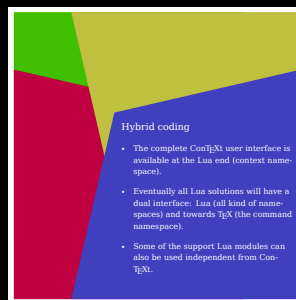
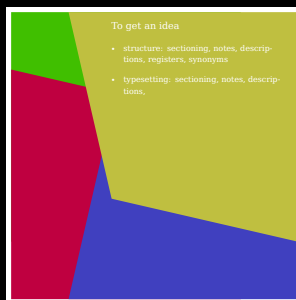
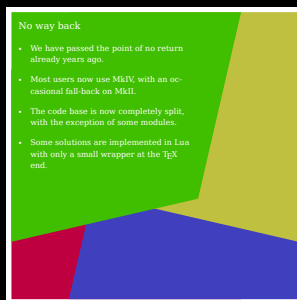
bachotex-2010-clash



bachotex-2010-move



bachotex-2011-cld-and-mkvi



bachotex-2013-bits

Bits and pieces:

ConTeXt
MetaPost
Lua and more

Hans Hagen
EuroBachOT_εX
May 2013

ConTeXt recently done	ConTeXt next on the todo list	MetaPost recently done
MetaPost next on the todo list	Lua recently done	Lua next on the todo list
LuaTeX recently done	LuaTeX next on the todo list	Fonts next on the todo list
Manuaals	Scripts	Speed

ConTeXt
recently done

- some more cleanup of old left-over
- most mechanisms now use the new level of parameter abstraction
- only a few fundamental incompatibilities (right of mechanisms, more control)
- slow introduction of hooks and extensions via setups
- first version of new multi-column routines
- replacing mechanism that have a (too) complex implementation
- exploring the mix (e.g. chemistry with Alan)
- remove some ugly left-overs from MetaPost
- normalized dynamic fonts (possibly interfacing)

ConTeXt
next on the todo list

- more definitive split between generic and context (generated)
- some math extensions, maybe already math dictionaries
- normalize all styles and modules
- play a bit more with the Lua parser/loader
- add more integrated bib layout support
- finalize experimental (auto)script code
- investigate what more is needed in the cxx interface
- check multi-lingual interface translations (not entirely in sync now)

bachotex-2013-luatex

LuaTeX for dummies

(so you can still leave)

Hans Hagen
EuroBachOT_εX
May 2013

The TeX perspective	The Lua perspective	The hybrid perspective
The complications	The future	This workshop
Just plain	A bit less plain	Hardly plain

The TeX perspective

- it started out as pdfTeX
- then got merged with AlgaTeX
- but we left out the ugly bits of both
- its exclusively utf-8
- its math machinery got extended with OpenType like features
- there are no fundamental extensions as its impossible to agree in them

The LUA perspective

- it just a Lua engine
- it has some extra libraries on board
- you don't even have to use TeX
- but there are hooks into the TeX machinery
- and we can go further by loading libraries

bachotex-2013-sense

How about those
typographic virtues:
do they still make sense?

Hans Hagen
EuroBachOT_εX
May 2013

Typesetting	In the process	But eventually
Targeting paper	Towards displays	The state of affairs
Take ligatures	Accent battles	Justification
Backslashes	Endangered features	The future of TeX

Typesetting

- somehow we turned sounds into speech into language
- and after that it may have started with writing in the sand
- followed by painting on cave walls
- or maybe carving symbols in wood
- and figuring out some writing system
- that made it possible to chisel thoughts in stone
- and eventually putting blobs on paper
- that we somehow managed to map onto pixels
- but
- typesetting only happened very lately

In the process

- markup optimized the basic shapes (to suit the language)
- and kept inventing new symbols
- more and more automated typesetting and rendering and printing
- and of course this went with all kind of silly arguments
- think of this:
- for ages scribes were the only way to get high end arabic typesetting
- and now we can do this mostly automatic (in good and bad ways)
- so we endangered yet another craft

bachotex-2013-speed

Speed:

can we make
it any faster

Hans Hagen
EuroBachOT_εX
May 2013

Speed

Pages per minute

What happens

What we can do

Speed

- speed matters in a add-on-preview cycle although this is mostly perception
- the nicer the interface, the slower it gets, but you seldom set something up
- everything you provide gets used at some point, also in inefficient ways
- lots of local (grouped) tweaks leads to many mechanisms kicking in unseen
- wrong use of functionality can have drastic and unexpected speed penalties

Pages per minute

- we try to speed up baseline performance (in pages per second)
- identify and optimize critical routines, both at the T_εX and L_α end
- of course the machine (Dell M99, SSD, 8GB, 2.33 GHz T7600, Windows 8) and version of L_αTeX (0.75) and ConTeXt (2.0)

# pages	January	April	May	(2013)
1	2	2	2	
10	15	17	17	
100	80	100	110	
1000	185	231	259	
10000	215	258	260	

bachotex-2015-ligatures

the interaction between
ligatures hyphenation
and kerning

Hans Hagen
BachOT_εX 2015

Ligature

effe
effe
effe

Discretionary

`\discretionary{pre}{post}{replac}`
explicit: \-
automatic: -
regular: patterns
first: internal
second: internal

Hyphenated ligature

ef fe
e [f] [f] e
e [f] [f] e

bachotex-2016-opentype

OPENTYPE FONTS the generic loader

Hans Hagen - bachOT_εX 2016

how engines sees a font

`\font`
fields: x-height, height, depth, table correction, kern table, ligature table, of commands, user data pointers, extensible
specifications and set of user and engine parameters
`\fontdimen`
extra fields: left processing, right processing, expansion factor and parameters to control these
`\fontfamily`
extra fields: width register, shift for ascenders, descenders, aligned extensible specifications, vertical capheight,
horizontal capheight, kern table, kern table, kern table and extra parameters and user parameters and set of
font functions

`\fontspec`
probably something similar

O

font handling

loading opentype font data

- all recently we used the built-in fontspec loader library
- but now we use a second system for loading
- but use a smaller fontspec loader
- but use a smaller fontspec loader

applying opentype features

- generic features: kern, table
- ConTeXt features: kern, table, kern, table, kern, table

loading opentype fonts

- the font is generic, loaded in ConTeXt
- user can use generic, loaded in ConTeXt, different in T_εX
- user can use generic, loaded in ConTeXt, different in T_εX
- user can use generic, loaded in ConTeXt, different in T_εX

OP

preparations

after loading

- initialize fontspec loader
- initialize fontspec loader
- initialize fontspec loader
- initialize fontspec loader
- initialize fontspec loader
- initialize fontspec loader
- initialize fontspec loader
- initialize fontspec loader

fontspec

- efficient access to all font properties for additional processing beforehand or afterwards

fontspec

fontspec

fontspec

fontspec

fontspec

fontspec

fontspec

fontspec

fontspec

fontspec

fontspec

fontspec

fontspec

fontspec

Grandpa's toolbox

(making closets)

Dad's hobby

(cleaning closets)

Uncle's friends

(talking closets)

bachoT_EX 2016

-

+

- there's closets and closets
- take the ones you put stuff in ... e.g. that you buy at ikea
- you have to assemble them so you get out your toolbox
- this kid sits next to you wondering what that tool is
- what is natural for you to use might not be so for them
- but grandpa likes to carry over his knowledge and experiences

-

closets

+

- but ... kids get their information from the internet, not from you
- (they watch discovery channel or national geographic and know a lot)
- or they look at vloggers (no bloggers) trying to learn something
- and they keep moving on ... and on
- do gp's live long tools really make sense to them?

-

interest

+

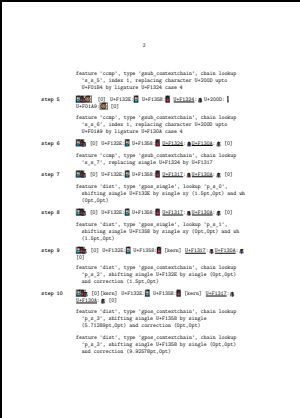
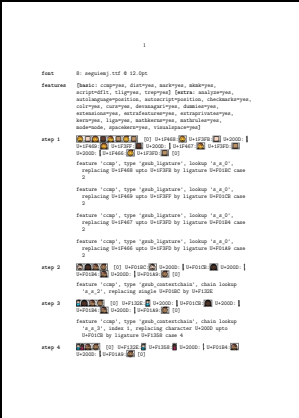
- grandpa also has a vlogger, he is called knuth
- gp is as locked into tex as the kids are into apps
- he's a do-it-him-self kind of person
- will his grandkids love watching him crafting
- and hear his stories about meetings and journals

-

grandpas

+

bachotex-2017-emoji-demo



bachotex-2017-emoji

Picture Fonts

welcome to a (beautiful) mess

Hans Hagen
BachTUG 2017

A Summary

- the **macro package's view**: just a font like any other but it needs to configure some extra color related properties
- the **engine's view**: depending on the technology a normal font that needs a bit special treatment or needs to be dealt with an collection of graphics
- the **viewer's view**: regular outline glyphs or images tagged as kind of characters so that their unicode representations can be cut and paste
- the **user's view**: more pictures than glyphs although some people one can communicate using them

So, in practice, for most $\TeX$ users it's probably not a high priority font but more a fun one.

Technologies

As each vendor came up with something, we have to deal with a all kinds of formats. And of course, as exerpases pushes things on the market before it's perfect we now have to deal with all of them.

- overlapping glyphs**: this technique uses the `colr` and `cpl` tables and is actually a quite clean technology you can combine in different ways
- svg graphics**: this technique uses the `svg` table that contains a `svg` vector image
- bitmap graphics**: this technique uses for instance this tables that can have various graphic images

The first two are already supported in the `CONTEX` font loader and processor for a while, the last one was added recently.

Only the overlapping method is useable for the tens of thousands of skin tone combinations of families, (kissing) couples, and professions.

Preparation

For now one has to enable the feature:

```
\definefontfeature{overlay}[default]{ccp=yes,col=yes,dist=yes}
\definefontfeature{svg}[default]{svg=yes}
\definefontfeature{bitmap}[default]{sbix=yes}

\definefontfeature{colored}[default]{
  [ccp=yes,dist=yes,
  col=yes,svg=yes,sbix=yes]}

Defining a font is not different from others
\definefont{MyEmojiFont}[sequence]{overlay}
\definefont{myemoji}[sequence]{sequence}{overlay}

As is using:
{\MyEmojiFont}{resolvedemoj(woman)}
{\emoj(woman)}
```


bachotex-2018-fonteffects

**Modern
Latin**

BachOT_gX 2018 — Hans Hagen

Why oh why

- In ConTeXt we have a mechanism to apply effects to a glyph stream.
- An active user on the ConTeXt mailing list wondered if that could be applied to specific fonts.
- The particular interest concerned the possibility to bolden fonts.
- I don't really like effects and not all fonts are suitable for it.

What are effects

Normal effects are implemented using the 'effects' mechanism, which already dates way back in MetaPost times and is also available for MetaX.

```
def\boldfaceeffect
\setfont{serif}
\setfont{bold}
\setfont{serif}
\setfont{bold}
\setfont{serif}
\setfont{bold}
```

effect effect effect
effect effect effect
effect effect effect

inner outer both

bachotex-2018-mp

**MetaPost
Extensions
A few examples**

BachOT_gX 2018 — Hans Hagen

History

- We started using MetaPost some time ago and immediately used the pdf version.
- We used special colors plus special to communicate extension, for instance each color and shape.
- This mechanism was simple and extended. Some mechanisms, like text, needed an extra pass.
- When we moved to LuaTeX and again we started using pre- and postscripts to easily communicate with the public.
- Currently we use a lot of Lua from within able to communicate during the MetaPost run with LuaTeX. This provides better interaction.

Colors

```
def\setcolor
\setcolor{red}
\setcolor{blue}
\setcolor{green}
\setcolor{yellow}
\setcolor{cyan}
\setcolor{magenta}
```

bachotex-2019-followingup

**The follow up
LuaMetaT_gX**

BachOT_gX May 2019

Hans & Alan

History

- This talk is not about how we can use LuaTeX to extend document applications. We already discussed that.
- This talk is not about how we can use LuaTeX to extend document applications. We already discussed that.
- This talk is not about how much fun it would be to have a MetaPost extension and use what ConTeXt and LuaTeX could do with it. We just can't afford it.
- This talk is not about more complexity, but it is about keeping things simple. It's about turning a burden into a pleasure.
- To quote the Riverside frontman: I hope you all have here a lot stronger than you felt when you came here. This talk is about turning lead into gold.

LuaMetaT_gX
From lead to gold

Colors

- We have the benefit and experience of about 40 years of TeX and its usage on our shoulders.
- Good old TeX got extended. TeX (Ample), TeX (Ample), TeX (Ample) and TeX (Ample) showed up.
- The dot output got complemented by pdf.
- Binary fonts were replaced by Type 1 fonts that could be replaced by the container format OpenType and TrueType. Variable fonts were introduced.
- MetaPost got updated to OpenType math, thanks to M. Reus.
- Unicode got accepted and still is nowadays the preferred input encoding.
- The community supported the development of many fonts that found their place in documents.
- Alternative plain TeX-like macro packages like TeX and TeX-like both evolved into large collections of resources.

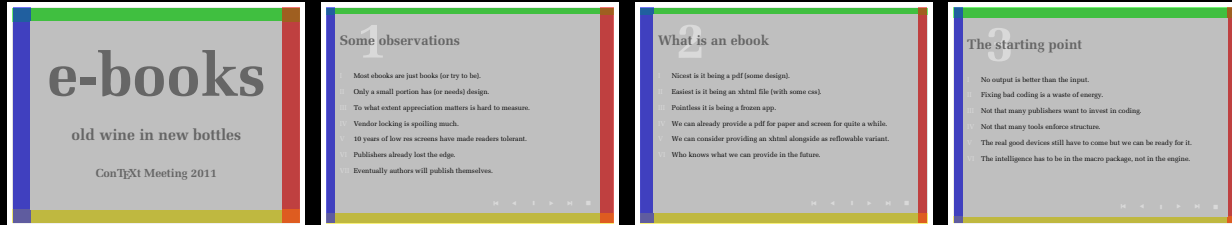
LuaMetaT_gX
Where do we stand

Colors

- There is no doubt that TeX is a success. We can find new examples but also some horrible looking documents.
- A large distribution is no guarantee for quality and continuous success, nor is the number of academic research users.
- A ConTeXt user doesn't need that much, just the TeX-like binary will do, plus a bunch of MetaPost, compiled by a reasonable set of fonts.
- Currently all that is embedded in a large extension, although we always use only a small, reasonable subset.
- Getting the whole machinery up and running from scratch is never easy and is not trivial.
- The source code base is rather large and compilation is complex. It builds on decades of heavy work for all platforms and facilities all demands.
- What we consider gold could also be seen as lead in disguise. Some of heavy might be needed to go back to where we came from.

LuaMetaT_gX
Are we good?

context-2011-ebook-export



context-2011-mathml-update



context-2011-metapost-how-we-adapt



1

Sorting registers

ConTeXt Meeting 2011

2

The old way

in M&T sorting is delegated to TeXfont i.e. a multipass action
 encoding vectors are passed along
 sort vectors depend on the language
 there are the usual complications with direct characters and commands

3

Moving on

in M&T sorting happens during the run
 we only have to deal with Unicode (utf)
 sort vectors still depend on the language
 sorting can be controlled by methods
 there is no universal solution (conflicting user demands, mixed languages)

4

Character data

unicode 0	unicode 1	unicode 2	unicode 3	unicode 4
abcode 0	abcode 1	abcode 2	abcode 3	abcode 4
abcode 5	abcode 6	abcode 7	abcode 8	abcode 9
abcode 10	abcode 11	abcode 12	abcode 13	abcode 14
abcode 15	abcode 16	abcode 17	abcode 18	abcode 19
abcode 20	abcode 21	abcode 22	abcode 23	abcode 24
abcode 25	abcode 26	abcode 27	abcode 28	abcode 29
abcode 30	abcode 31	abcode 32	abcode 33	abcode 34
abcode 35	abcode 36	abcode 37	abcode 38	abcode 39
abcode 40	abcode 41	abcode 42	abcode 43	abcode 44
abcode 45	abcode 46	abcode 47	abcode 48	abcode 49
abcode 50	abcode 51	abcode 52	abcode 53	abcode 54
abcode 55	abcode 56	abcode 57	abcode 58	abcode 59
abcode 60	abcode 61	abcode 62	abcode 63	abcode 64
abcode 65	abcode 66	abcode 67	abcode 68	abcode 69
abcode 70	abcode 71	abcode 72	abcode 73	abcode 74
abcode 75	abcode 76	abcode 77	abcode 78	abcode 79
abcode 80	abcode 81	abcode 82	abcode 83	abcode 84
abcode 85	abcode 86	abcode 87	abcode 88	abcode 89
abcode 90	abcode 91	abcode 92	abcode 93	abcode 94
abcode 95	abcode 96	abcode 97	abcode 98	abcode 99
abcode 100	abcode 101	abcode 102	abcode 103	abcode 104
abcode 105	abcode 106	abcode 107	abcode 108	abcode 109
abcode 110	abcode 111	abcode 112	abcode 113	abcode 114
abcode 115	abcode 116	abcode 117	abcode 118	abcode 119
abcode 120	abcode 121	abcode 122	abcode 123	abcode 124
abcode 125	abcode 126	abcode 127	abcode 128	abcode 129
abcode 130	abcode 131	abcode 132	abcode 133	abcode 134
abcode 135	abcode 136	abcode 137	abcode 138	abcode 139
abcode 140	abcode 141	abcode 142	abcode 143	abcode 144
abcode 145	abcode 146	abcode 147	abcode 148	abcode 149
abcode 150	abcode 151	abcode 152	abcode 153	abcode 154
abcode 155	abcode 156	abcode 157	abcode 158	abcode 159
abcode 160	abcode 161	abcode 162	abcode 163	abcode 164
abcode 165	abcode 166	abcode 167	abcode 168	abcode 169
abcode 170	abcode 171	abcode 172	abcode 173	abcode 174
abcode 175	abcode 176	abcode 177	abcode 178	abcode 179
abcode 180	abcode 181	abcode 182	abcode 183	abcode 184
abcode 185	abcode 186	abcode 187	abcode 188	abcode 189
abcode 190	abcode 191	abcode 192	abcode 193	abcode 194
abcode 195	abcode 196	abcode 197	abcode 198	abcode 199
abcode 200	abcode 201	abcode 202	abcode 203	abcode 204
abcode 205	abcode 206	abcode 207	abcode 208	abcode 209
abcode 210	abcode 211	abcode 212	abcode 213	abcode 214
abcode 215	abcode 216	abcode 217	abcode 218	abcode 219
abcode 220	abcode 221	abcode 222	abcode 223	abcode 224
abcode 225	abcode 226	abcode 227	abcode 228	abcode 229
abcode 230	abcode 231	abcode 232	abcode 233	abcode 234
abcode 235	abcode 236	abcode 237	abcode 238	abcode 239
abcode 240	abcode 241	abcode 242	abcode 243	abcode 244
abcode 245	abcode 246	abcode 247	abcode 248	abcode 249
abcode 250	abcode 251	abcode 252	abcode 253	abcode 254
abcode 255	abcode 256	abcode 257	abcode 258	abcode 259
abcode 260	abcode 261	abcode 262	abcode 263	abcode 264
abcode 265	abcode 266	abcode 267	abcode 268	abcode 269
abcode 270	abcode 271	abcode 272	abcode 273	abcode 274
abcode 275	abcode 276	abcode 277	abcode 278	abcode 279
abcode 280	abcode 281	abcode 282		

Breakdown 2012	After the change	Breakdown 2012	After the change	Breakdown 2012	After the change	Breakdown 2012	After the change
Update - The move to MVC is more than supporting an engine. - It is a complete rewrite (grammar, windowing, classes). - Although somewhat crippled by the fact that we want to remain compatible. - But sometimes we sacrifice compatibility by getting rid of old stuff.		What is there to do - Some code might become generalized (also depends on others). - Layer and partitioning code might get a more extensive class and sub interface. - Structure related code will suggest some more code to be added (in depth). - New page header variables will be explored (extension class and some support and floats). - Math domain eq. distribution (linear ahead) to allow, not needs time. - Math list optimization (get). - Common dependency (more linear) and more consistent code building order. - Event message needs checking (some gone, some not yet translated). - Update all definitions (work in progress, also related to which). - Optimize partitioning system (it will be more powerful now, but also more resource). - More support for the code (the thing I think I want to do). - Optimize (especially those for tracing) need to be investigated. - Some topics (mostly geometry partitioning) (also needs to be fixed). - Pick up the "ben and moor" (C/C++ variant) project. - Now, that we have more code instead, we can define an app. - Some resources need to be updated (most still apply).		What I have to keep in mind - What is handy for me is not always handy for all users. Not everything there will be new things - Elements of my previous framework will show up in the distribution. - It's just more consistent to have one initialization for related things. - This is also why support for decisions has been added recently. - Running (building) to the code needs special treatment (fixed management). - It makes sense to use the old developed test infrastructure.		Keep an eye on updates - Reverting the code base leads to keep but these are often reversed quickly (induced by Web team). - Changing the building logic and normalize the logic in adapted class to realize (inspired by Socrates). - Changes in standards and related tools are supported and followed (those who depend on them (ask Peter)). - Changes in areas, have demands and these end up as extensions to existing mechanisms (Aditya's whole mission). - Changes with platform are often quickly dealt with (of Long doesn't know...). - And of course I add new things driven by projects, challenges (and I can make a request for that). - Now releases (and beta) are characterized by a growing set of new files (this includes more or less each update).	

Breakeins 2012	Learning resources	Breakeins 2012	Learning resources	Breakeins 2012	Learning resources
What is lexing? - Computer languages have mandatory structures. - You can avoid errors by checking the input. - Editors can help by offering reserved words, context etc. - Consistency in coloring different language tokens seems.		Side effect of MKEY - SCITE got LEXC based lexing (internal lexing). - I already had already written some lexers for the pretty printers. - So I guess it is up and runs on some more advanced lexers. - Those ship with CONTEXT, QA, XML, PWF, LHA, CLD, METANODE, best.		The future - I will improve the current lexers. - An SQL lexer will be added at some point. - I might make an SYNTAX-CLD variant that supports nested lists.	
When did we start - We wrote our first editor beginn 90's. - An extension quickly followed when we moved to SCILAB, THERMIST. - When METACAL was no longer feasible we moved on to SCITE, XMOORE (quick down) - When we ran into SCITE we start using that. - I provided system building for QA and METANODE (support for multiple formats etc.).		Characteristics - The QA lexer supports nested lexing of LHA and METANODE. - Integrated spell checking is provided. - Unfortunately there is no lexing for SCITE on MACOSX (not that I care too much nowadays). - It is a pity that we have no access to SCITE internal as well the regular UI interface. - On my old and machine huge files are sometimes slow (at the end).			

context-2012-mixed-columns

Breakdown 2012 The script

Output

- `ly` collects content paragraph wise.
- In between it can trigger the so called output routines.
- At first moment you can do something with the results.
- One of the things you can do is package all collected so far in a page.

Boxes

- In `lualatex` we can also intercept content at some places.
- For instance before and after each paragraph is processed.

Breakdown 2012 The script

Columns

- `ly` has no concept of columns.
- You need to fake them by filling with the width and spilling boxes.
- Often we run our `TeX` code (no output content).
- For some local applications we use simple columns.
- In far instance `TeX` can use a mixed size and multi-column model.
- Columns are another (independent) mechanism, strongly grid based.
- Traditional multi-columns are being replaced by a new mechanism: mixed columns.

Pitfalls

- Fontsize: page, first or last column, each column (aligned, immediate).
- Graphics: mixing floats around is more complex than in single columns.
- Nesting: how about columns inside columns.
- Relinking: can be hard taking all into account.

Breakdown 2012 The script

Questions

- Do complex column mechanisms still make sense given the move to electronic paper.
- If so, what functionality should be provided.

context-2012-the-script

Breakdown 2012 The script

Some myths

- `CONTEXT` looks at lot like plain `TeX` and expects users to program macros.
- `CONTEXT` depends on `TeX`.

The truth

- On the average users don't have to program. ConTeXting is not programming.
- As `TeX` lacks communication handling and job-control, helpers are provided.
- All context users can still program a lot, but not all need that.
- All context users can directly run `CONTEXT`, but why should they?

A few facts

- The `CONTEXT` distribution provides a sort of ecosystem.
- In `TeX` indeed we use `TeX` for some job control.
- But in `TeX` all is left context done in `TeX`.
- Two scripts play an important role: `mkx` and `context`.

Breakdown 2012 The script

The 'mkx' script

- Locates and runs scripts, has a lot of helpers predefined.
- It is in fact the `TeX` runner on top the `ly` script.
- It knows about files and the environment we run in.
- It has some features that makes it easier to integrate in services.
- The way we don't need much (and avoid potential conflicts in some).

The 'context' script

- It runs `CONTEXT` and keeps track of how many runs are needed.
- Contrary to its `TeX` ancestor it is not needed for make testing etc.
- It has a few extensions that are loaded on demand: `extra`.

Breakdown 2012 The script

A regular run

```
context [-:run] filename
```

Running from an editor

```
context --script filename
```

Running from an service

```
mkxrun --path=filepath --script context filename
```

Breakdown 2012 The script

Controlling the rendering

```
--noheaderlist
--nofooterlist
--noindexlist
--noindexlist
--noindexlist
--noindexlist
```

Controlling with `ctx` files

```
--ctxname
```

Also in preamble

```
<?context-directive job stylefile email.ctx ?>
```

context-2012-visual-debugging

Breakdown 2012 Visual debugging

How it started

- Some 15 years ago I wanted some more feedback.
- First I started out a way to translate boxes, lines, etc.
- Some aspects were tricky, like stretch and shrink (as a `TeX` unit), fillers, leaders, etc.
- I gave some presentations and it was not to see the parallel faces.
- As debugging does not work, it is somewhat interfering.
- When not needed there is no overhead but we did think it at some places.

Do we need it

- I wonder if anyone ever used it.
- Some of the helpers are quite handy, like `ly` and `lualatex`.
- So there had to be provided anyway, so: when to stop?

Breakdown 2012 Visual debugging

All kind of debugging

- We have some debugging, much shows up when writing new code.
- Think of fonts, math, graphics, characters, etc.
- Some make us aware in `TeX`, so they're gone, but some more show up.
- In that time there will all be surrounded (or more) lines in modules.

Breakdown 2012 Visual debugging

Why we kept it

- When cleaning up the code I had to decide to keep it or redo it as it could be done `ly`-ish.
- But as we already had some `ly`-based extra it made sense to redo it.
- The old code is still there as module (also because it had some more useful).

How it worked

- In `TeX` processes are overlaid.
- So effectively when enabled, `Visual` can run become macro.
- We use rules (and leaders) to window properties.
- Some constructs interfere so we need to compensate side effects.

Breakdown 2012 Visual debugging

How it works

- The helper was a rather trivial quick job as we had a lot in place already.
- Interpreting the mode list and existing structures.
- The new colors, rules and text box stuff can be overlaid.
- Control over what gets rendered at the `ly` end.
- Control over what gets shown by using layers.
- A second mode time more into mechanism, choices and optimization.
- Some structures interfered with (hard-coded) expectations in the backend.
- When I decided to run layers I had to adapt some other code (mostly out of `offscreen`).
- There is room for more (that first I want the bits of `ly` 5.2).

context-2012-xml-news

Weeks 2012	Processing XML, some basics
Topics	
• processing	
• selecting	
• flushing	
• testing	
• basic only	

```

Processing
  \setprocessfile (name) (filename) (setup)
  \setprocessbuffer (name) (filename) (setup)
  \setloadunit (name) (filename) (setup)

Loading
  \setload (name) (filename) (setup)
  \setloadbuffer (name) (buffername) (setup)

```

```

Breakdown 2012
Processing XML, some basics

Injecting elements

<valcell (mode) (pattern)>
<vallive (mode) (pattern)>
<vallast (mode) (pattern)>
<valfund (mode)
<br>
<valrow (mode) (pattern)>
<valcontact (mode) (pattern)>
<validip (mode) (pattern)>
<valing (mode)
<valtaxi (mode) (pattern)>

```

```

Breakdown 2012
Processing XML, some basics

Injecting attributes

\xmlatt (node) (name)
\xmlattf (node) (name) (default)
\xmlattribute (node) (pattern) (name)
\xmlattributef (node) (pattern) (name) (default)

Injecting properties

\xmlcontent (node) (pattern)
\xmlname (node)
\xmlnamespace (node)

```

context-2013-math

Math:

progress or standing still

Hans Hagen
ConTeXt Meeting
September 2013

Math as script	Alphabets	Heavy bold	Radicals
Primes	Accents	Stackers	Fences
Directions	Structure	Italic correction	Big
Macros	Unscripting	Combining fonts	Tracing

- math can be input using the TeX system. Monodiff, calculate like equations...
- but output from control Monodiff, all stay close to good old TeX
- although not officially a script, OverTeX treats it as such, but without control

```

B (a - x) / a + 1/2 = (a - x) / b B
B \left( \frac{a-x}{a} + \frac{1}{2} \right) = \frac{a-x}{b} B
\documentclass{beamer}
\usepackage{amsmath}
\begin{document}
\begin{equation}
\frac{a-x}{a} + \frac{1}{2} = \frac{a-x}{b}
\end{equation}
\end{document}

```

There is recognition of math as a proper (but not standardized) script.

Alphabets

- the shape (style) of a character determines its meaning
- but in most cases an type is a mirrored as-sorts character
- and tagged with some rendering directives, often indicating a font style
- in traditional T₈ we have alphabets in different fonts, so we're talking overviews
- in TeXmacs and CTeXType we have alphabets with standardized code points (but gaps too)
- this has big advantages for communicating, transferring data etc
- but a much bigger still has to deal with a-sorts input as well
- multiple axis: types, alphabets, styles, variants, shapes, modifiers

We're off better but the gaps are an anomaly:

context-2013-speed

Speed:

why it matters
and why we care

Hans Hagen
ConTeXt Meeting
September 2013

Speed	Pages per minute
What happens	What we can do

Speed

- speed matters in a solid-run preview cycle although this is mostly perception
- the more the interface, the slower it gets, but you seldom set something up so that is not much of a burden
- everything you provide gets used at some point, also in inefficient ways, so don't leave your weak spots
- initial load (grouped) breaks leads to many mechanisms kicking in, unseen, grouping matters
- writing use of functionality can have drastic and unexpected speed penalties

Pages per minute

- we have speed up the baseline performance (in pages per second) as much as possible
- we try to identify and optimize critical routines, both at the TgN and LoG end
- of course the used hardware makes major versions of LoG-TgN and CoG-TgN matter

Videorecas (1000) (real usage)

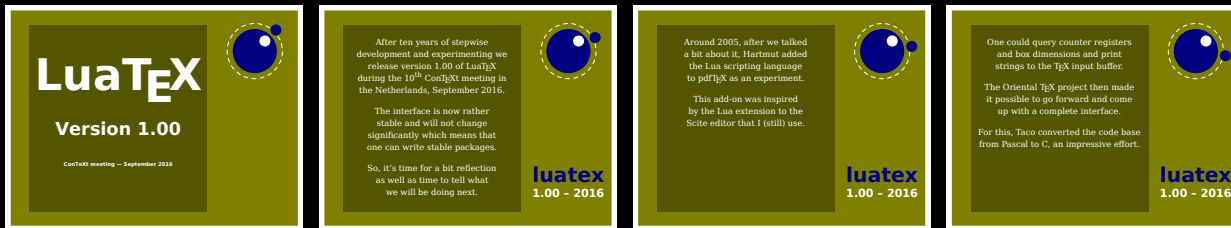
Pages	Jan	Apr	May	Sept	min
	2	2	2	4	4
10	15	17	17	36	37
100	80	109	110	237	236
1000	185	234	259	509	512
10000	212	259	299	540	540

7/96/2013, LoG-TgN v0.747, LoG-MST v0.553, 4GB, 2.33 GHz T7400, Windows 8/32 bit
 > 06/2013, LoG-TgN v0.722, LoG-MST v0.553, 4GB, 2.80 GHz SR04QM, Windows 8/64 bit

context-2015-status



context-2016-luatex



context-2017-css-selectors

CSS selectors	
Needed or not?	Simple formulas
Supported methods	Special cases
Filtering classes	Combinators
Filtering attributes	Composers (1)
Combining methods	Composers (2)
Child selectors	
CONTyXV 2017 Malbarch	

- used in `ITIME` styling
- a bit different from the path based method
- shortcuts for filtering by attribute
- class filtering is special because it checks for list
- one can select more at the same time
- performance is okay compared to path lookup
- selectors go between curly braces

```
unfall (#) {foo bar .whatever, bar fun .whatever}}
```

The following methods are supported:	
<code>element</code>	all tags element
<code>element-1 > element-2</code>	all tags element-2 with parent tag element-1
<code>element-1 < element-2</code>	all tags element-2 preceded by tag element-1
<code>element-1 ~ element-2</code>	all tags element-2 preceded by tag element-1
<code>element-1 & element-2</code>	all tags element-2 inside tag element-1
<code>[@attribute]</code>	has attribute
<code>[attribute=value]</code>	attribute equals value
<code>[attribute!=value]</code>	attribute contains value (space is separator)
<code>[attribute="value"]</code>	select with value
<code>[attribute~"value"]</code>	attribute may with value
<code>[attribute "value"]</code>	attribute contains value
<code>~</code>	has same
<code>#id</code>	has id
<code>child:child1</code>	the child of node n
<code>child-last-child(n)</code>	the last child of node n
<code>first-child</code>	the first child
<code>last-child</code>	the last child
<code>nth-of-type(n)</code>	the nth child of node n
<code>nth-last-of-type(n)</code>	the nth child of node n from the end
<code>nth-of-type</code>	the first match
<code>last-of-type</code>	the last match
<code>only-of-type</code>	only the match or nothing
<code>only-child</code>	only the child or nothing
<code>empty</code>	empty
<code>/not</code>	the whole tree

[illegible]

context-2017-features-chaintest



context-2017-features-kerntest



context-2017-features-pairtest

A B C D E F . . .

context-2017-features-singletest

A B C D E F . . .

context-2017-features-spacetest

A B C D E F G H . . .

context-2017-features-substitutiontest

abracadabra
XPrXcXdXPrX
YQrYcYdYQrY
XYPQrXYcXYdXYPQrXY
1raca2bra
aBraCadaBra
effe 1b3def !f . . .

context-2017-features

Font features

What are they
Substitution
Positioning
Ligatures
Pseudo
Examples

CON_TE_XT 2017 Mailbach

- built in substitution that is often optional like ligatures but for some languages mandatory
- built in positioning that is assumed to be applied like kerning, mark anchoring, etc.
- external properties like coloring, spacing, fallback combinations
- engine related tricks like expansion and protection
- tracing options
- whatever you like ... so let me know
- so in CON_TE_XT we have font features (driven by font) and pseudo features (driven by additional needs)

What are they

- single: replace one by another
- alternate: replace one by one of a set
- multiple: replace one by multiple others
- ligature: replace multiple by one shape
- contextual lookups and replacements with look back and look ahead

Substitution

- single: repositioning a glyph (with optional marks), this includes traditional kerning
- pairwise: repositioning two adjacent glyphs (with optional marks)
- anchoring: often used for marks to base glyphs, ligatures and other marks
- attachment: often used for various scripts, putting glyphs in a word together
- contextual lookups and positioning with look back and look ahead

Positioning

context-2017-performance

Performance

Why bother
Possible bottlenecks
Measurements
Examples

CON_TE_XT 2017 Mailbach

- because we don't want to waste time waiting
- because I get tired of ignorance related complaints
- because it (sometimes) can be a nice puzzle to improve performance
- because it occasionally reviewing makes code better

Why bother

- starting up
- loading fonts
- processing features
- applying trickery
- enabling tracing
- fonts
- L^AT_EX
- images

Possible bottlenecks

- how long does a run take
- how does the number of pages matter
- how many runs are needed
- start-up time
- processing pages
- finishing the document
- the console used

Measurements

context-2017-synctex

SYNCTEX

What is SYNCTEX
Disclaimer
What we do
How it works
What the user gets

CON_TE_XT 2017 Mailbach

- it is a mechanism for going back from viewer to editor
- it uses an extra (cached) output file
- it adds an overhead of 2 to 15 percent runtime
- it is designed with a specific macro package in mind
- the rather generic approach works okay for simple document layouts
- but it often fails for projects that use multiple files
- and that moves information around like X_YL recorded files

What is SYNCTEX

- all recently it was supported in CON_TE_XT as it was
- there were no complaints, so it must have worked ok for most users
- we never used it ourselves because of mentioned reasons
- we only wanted to support it when it works ok in projects
- (think of thousands of X_YL with deeply nested inclusions in one document)
- but what we support now is purely based on personal experience
- we don't use it ourselves so feedback is welcome

Disclaimer

- the internal SYNCTEX mechanism is disabled
- when told so, CON_TE_XT will look in its own code
- this is done by using L^AT_EX code to set the right information
- only source files that make sense are dealt with
- this protects the styles from unwanted changes
- within reasonable bounds X_YL is supported
- this also includes nested documents

What we do

```

graph LR
    subgraph Tables_Box [Tables]
        direction TB
        T_Variants[Variants  
tblr  
tabulate  
natural tables  
extreme tables  
low tables  
frame tables]
        T_Context[CONTEXT 2017 Mailbach]
    end

    subgraph Variants_Box [Variants]
        direction TB
        V1[■ Good old \tblr = wrapper about \X's alignment.]
        V2[■ Running text, break across pages tabulates.]
        V3[■ Tables that behave like HTML, tables therefore called natural tables.]
        V4[■ A variant on this that is more easy to extend, tagged extreme tables.]
        V5[■ A low profile installable mechanism that can open pages and breaks well.]
        V6[■ A way to make huge tables without overflowing \X too soon.]
    end

    subgraph tblr_Box [tblr]
        direction TB
        T1[■ Based on the \tblr macro package.]
        T2[■ Detailed control over spacing.]
        T3[■ Somewhat inconsistent spacing out of the box.]
        T4[■ Extended with extra features.]
        T5[■ Mostly rewritten but within the original concept.]
    end

    subgraph Tabulate_Box [Tabulate]
        direction TB
        T6[■ Mostly meant for tables that are part of the text flow.]
        T7[■ Breaks paragraphs across pages.]
        T8[■ Sort of compatible in control with \tblr.]
        T9[■ The system that I used most often.]
        T10[■ It uses multiple passes if needed.]
    end
  
```

Lean and mean

- interference:** LuaTeX, plainTeX and CTAN all have different demands (we want to experiment and more we can control path by way)
- consistency:** the source tree is way too complex as in the build (you only need LuaTeX)
- distributions:** no one can guarantee stability for LuaTeX (there is a more player that offers a bit already)
- annotations:** experimental code leads to bugs outside LuaTeX and that triggers complaints

How it became

- motivation:** reusing ideas from folks who know to express "more logic" and "more modularity" makes sense
- goals:** I got tired of "you need to express this because..." habits
- possibilities:** the danger of mixing Lua with the core and the possibility to reuse ideas from LuaTeX

Implications

- history:** there is only one relatively small history (that does not change rapidly)
- code base:** there exists an entry source tree, but it's not very well documented (2 MB)
- user control:** if needed users can compile the program as we wish contained
- future safe:** we can move forward and improve
- modernity:** a code base with the latest LuaTeX, LuaTeX and TeX (I'm satisfied)
- marketing:** this way the relation with LuaTeX and its users can be more clear
- playground:** we have a better separation between the development of running into problems with more LuaTeX users... "not at all new stuff"
- possibilities:** the danger of mixing Lua with the core and the possibility to reuse ideas from LuaTeX

[illegible]

ConT_FXt Meeting

context-2020-datatypes

DATATYPES

additional datatypes in lmtx

Native T_EX datatypes: simple registers

```
integer: /count223 + 456 /the/count223
integer: 456
dimension: /dimension223 + 456opt /the/dimension223
dimension: 456,opt
give: /skip223 = 4pt plus 5pt minus 4pttimes /the/skip223
give: 4.5pt plus 5.5pt minus 4.4pt
negate: /unskip223 = 4mu plus 5mu minus 4mu /the/unskip223
negate: 4.0mu plus 5.0mu minus 4.0mu
attribute: /attribute223 + 456 /the/attribute223
attribute: 456

/global /the /counted /dimension /skipped /unskipped /attribute223
```

```
\advance \multiply \divide \numexpr \dinxpr \glueexpr \muexpr
```

100

Native T_EX datatypes: tokens

```
toks: \toks123 = {456} \the\toks123
toks: 456

\global \the \toksdef
\toksapp \toksapp \xtoksapp \gtoksapp
\tokspre \tokspre \xtokspre \gtokspre

(On retrospect: notes)
```

context-2020-ecmascript

ECMAScript

just because it can be done

Why oh why

- We use a *mu2d* based pdf viewer: [SumatPDF](#).
- And occasionally we use the tools that come with *mu2d*.
- So when checking if that viewer supports JavaScript in widgets I noticed the stand alone interpreter:¹
- Which made me wonder how easy it would be to interface to it.
- It uses the lightweight library subsystem: like all the library interface is setup dynamically.
- Support is not integrated in `LuaMetaNGC`, so there is no overhead and there are no dependencies.
- We assume that the library is on the system, and when not, then there is also also no support.
- We stick to the absolute minimum of interfacing needed and delegate everything else to `Lua`.
- We assume a stable api, and if not, well ... sorry.

¹ The official name is *ecmaascript* which is the standardized core language.

The components

- The optional, delayed loading, interface, adds only a few KB to `LibMTeX2La`.
- The `La` library interface that is part of the `ConTeX2La` distribution which means that it's officially supported.
- There is a `TeX` module that loads the lot and provides the user interface.
- And of course, somewhere on the system, there should be the `mu` library.³
- A module like this should conform to the `ConTeX2La` L^AT_EX standards (a minimalistic not bloated API, interfacing in `La` and `TeX`, etc.).

In ConTeXt libraries go into the platform tree, like:

```
/tex/texmf-win64/bin/lib/luametatex/mujs/libmujs.dll
/tex/texmf-linux-64/bin/lib/luametatex/mujs/libmujs.so
```

² Taim compiled the library for his system during the talk and confirmed that it also works out of the box on an

An example

```
%usemodule([xmacscript])

%macrocode {
  comesc1e(""); comesc1e("when the loading has succeeded!");
  comesc1e("");
}

%macrocode {texprint{"Just a [\\bf short] sentence."}}

%startmacrocode
  %texprint{"And this is \\inframed{\\bb a bit longer} sentence."}
%stopmacrocode

Just a short sentence.

And this is a bit longer sentence.
```

context-2020-implementers

IMPLEMENTERS

an old feature still evolving

context 2020 meeting

Interfacing with Lua

- Quite some activity is delegated to Lua.
- Normally the initiative is at the $\mathcal{V}_{\text{LH}}$ end.
- We can set variables or call functions etc.
- We can parameterize to function calls.
- From the Lua end we can use scanners to pick up data.
- We provide some consistent interfaces for doing all that.
- From $\mathcal{V}_{\text{LH}}$ to Lua we use $\text{setLua}\{\dots\}$ and friends.
- From Lua to $\mathcal{V}_{\text{LH}}$ we use $\text{context}(\dots)$ and alike.
- For adding functionality we use so-called implementers.

Calling Lua

```

[ctoken|context("ok")]
ok
[ctoken|context(2 + token, scanners, Integer())] 10
20
[ctoken|unde
Function document.MyThing() context(2 + token, scanners, Integer()) end
[ctoken|unde
[ctoken|document.MyThing() 20] unde
[ctoken|document.MyThing() 30] unde
[ctoken|document.MyThing() 40] unde
80 60 80

```

Streamlining Lua

```

\startliteratecode
interface implement {
  name = "MyThing",
  public = true,
  arguments = "integer",
  actions = function(i) context[i + 2] end,
  -- actions = { function(i) return i + 2 end, context }
}
\stopliteratecode

\MyThing 20 \quad \MyThing 20 \quad \MyThing 40
40 60 80

```

ConT_FXt Meeting

LUAMETATEX
where do we stand
context 2020 meeting

journalist — context 2020 meeting — when it started

luametahe - context 2020 meeting - why it started

huametatex - context 2020 meeting - the development

MKII MKIV LMTX
where does it end
context 2020 meeting

METAFUN
simple fonts
context 2020 meeting

metafun - context 2020 meeting - metafun

metaphor — context 2020 meeting — what is is not

metaphor — context 2020 meeting — what is is not

ConT_FXt Meeting

context-2020-svg

SVG GRAPHICS

some demos and discussion

Wrapup

- It expands to Scalable Vector Graphics.
 - It is an example of application xml turned standard.
 - It started out simple, kind of expanded PostScript in xml format.
 - It took a while to be picked up as output format.
-
- In practice you get the same messy build-up as in other vector formats.
 - This is a side effect of often unstructured editing.³

¹ Afterwards Hrahan gave a demonstration of editing in landscape and there was some discussion about this aspect.

Properties

- Properties can be set as attributes to an element (key/values).
- Properties can be set in the style attribute (semicolon separated key/values).
- Properties can be set via one or more class assignments.
- Properties can be bound to a specific element
- Properties can be inherited from an ancestor (somewhat vague).
- Properties can be redundant (nested, overloaded (parent, style), editors can add their own, etc ... it's kind of a mess.

Side effects

```
\usemodule{gnuplot}

\externalfigure
{context-2020-gdmc.gp}
{conversion=svg,width=4cm,
background=color,backgroundcolor=white}

\externalfigure
{context-2020-gdmc.gp}
{conversion=svg,width=4cm,
background=color,backgroundcolor=white}

\scale
{height=4cm}
{\framed
{background=color,backgroundcolor=white}
{\includegraphics[width=file]{context-2020-sin.svg}}}
```

context-2020-tokens

TOKENS

tokens as I see them

About tokens

- Like nodes, it's a common term used in programming.
- In TPL The Program tokens and nodes are therefore omnipresent.
- For most users they are irrelevant concepts.
- But we will explain them anyway.
- Let's try to avoid the snobbish token-speak sometimes heard in the community
- So ... I won't correct you as long as you don't correct me.
- Let's now enter the world of tokens in the naive way.

What are tokens

- It is an internal data structure, effectively a 32-bit integer
- This integer encodes a command (opcode) and an char code (operand).
- But often if x isn't a character but more a sub command.
- Input is converted into tokens.
- Tokens are either expanded (interpreted) or stored.
- When they are stored they are part of a larger data structure, a memory word.
- Token memory is an array of such memory words.
- The token memory 'word' has two integers, a token value and an index into token memory.
- That way Tgk can have forward linked lists of tokens.
- A hash table maps current sequence into indices into token memory.

Some implementation details

- Sometimes there is special hand token at the start.
- A hand token makes for easier appending of extra tokens.
- Shared lists use the hand node for a reference count.
- Original TeX uses global temporary lists.
- This is needed when we expand (nested) and need to report issues.
- This is not needed when we just serialize (which we do a lot in Lua^{TeX}).
- So, this is all optimized for performance and memory consumption.
- Freed tokens are collected in a cache so tokens can get scattered.
- In LuaMetaTeX we stay as close to original TeX as possible.
- But the Lua interfaces force us to occasionally divert.

context-2021-compactfonts

COMPACT FONTS

a different approach to scaling

How T_EX uses fonts

- In a TeX engine there is a 1-to-1 mapping from characters in the input to a slot in a font.
- Combinations of characters can be mapped onto one shape: ligatures.
- Hyphenation is (also) controlled by the input encoding of a font which imposes some limitations.
- In the typeset result characters (glyphs) can be (re)positioned relatively: kerns.
- This all happens in a very intertwined way (e.g. inside the `par` builder).

How traditional T_EX sees fonts

- A font is just a binary blob of data with information where characters have a width, height, depth and initial c Kerning.
- The engine only needs the dimensions and when the same font is used with a different scale it copy with scaled dimensions is used.
- There can be recipes for ligatures and (more complex) so called math extensible shapes (like arrows, wave accents, braces and radicals)
- There are pairwise specification for kerning.
- A handful of font parameters is provided in order to control for instance spacing.
- Virtual fonts are just normal fonts but they have recipes for the backend to construct shapes $\Rightarrow$ use the normal matrix file, the backend also the (virtual) specification file.

An example of a definition and usage:

```
\font\ca somename [somescale] test {\ca test} test
```

Unicode engines

- There are no fundamental differences between a traditional engine and an Unicode aware one with respect to what *font* needs from font's dimensions.
- There can be more than 256 characters in a font.
- Some mechanisms have to be present for applying font features (like ligaturing and kerning).
- In LaTeX₂ hyphenation, ligaturing and kerning are clearly separate steps.
- In LaTeX₂ calligraphs can do lots of things with fonts and characters.
- Math fonts are handled differently and there is more info that comes from the font.
- In LaTeX₂ features like *expansion* are implemented differently, i.e. no copies of fonts are made and applied expansion travels with the glyphs.

[contact fonts](#) — [contact 2021 meeting](#) — [unicode guides](#)

PROGRAMMING

local control

context 2021 meeting

Expansion

Can be used as well as called using reading mode

Very easily seen: translating from the source file. Comments are picked up and interpreted. Depending on what token it becomes some action taken.

```
1 |var1=0;var2=10;|var3=0;|var4=0;|var5=0;|var6=0;|var7=0;|var8=0;|var9=0;|var10=0;|var11=0;|var12=0;|var13=0;|var14=0;|var15=0;|var16=0;|var17=0;|var18=0;|var19=0;|var20=0;|var21=0;|var22=0;|var23=0;|var24=0;|var25=0;|var26=0;|var27=0;|var28=0;|var29=0;|var30=0;|var31=0;|var32=0;|var33=0;|var34=0;|var35=0;|var36=0;|var37=0;|var38=0;|var39=0;|var40=0;|var41=0;|var42=0;|var43=0;|var44=0;|var45=0;|var46=0;|var47=0;|var48=0;|var49=0;|var50=0;|var51=0;|var52=0;|var53=0;|var54=0;|var55=0;|var56=0;|var57=0;|var58=0;|var59=0;|var60=0;|var61=0;|var62=0;|var63=0;|var64=0;|var65=0;|var66=0;|var67=0;|var68=0;|var69=0;|var70=0;|var71=0;|var72=0;|var73=0;|var74=0;|var75=0;|var76=0;|var77=0;|var78=0;|var79=0;|var80=0;|var81=0;|var82=0;|var83=0;|var84=0;|var85=0;|var86=0;|var87=0;|var88=0;|var89=0;|var90=0;|var91=0;|var92=0;|var93=0;|var94=0;|var95=0;|var96=0;|var97=0;|var98=0;|var99=0;|var100=0;|var101=0;|var102=0;|var103=0;|var104=0;|var105=0;|var106=0;|var107=0;|var108=0;|var109=0;|var110=0;|var111=0;|var112=0;|var113=0;|var114=0;|var115=0;|var116=0;|var117=0;|var118=0;|var119=0;|var120=0;|var121=0;|var122=0;|var123=0;|var124=0;|var125=0;|var126=0;|var127=0;|var128=0;|var129=0;|var130=0;|var131=0;|var132=0;|var133=0;|var134=0;|var135=0;|var136=0;|var137=0;|var138=0;|var139=0;|var140=0;|var141=0;|var142=0;|var143=0;|var144=0;|var145=0;|var146=0;|var147=0;|var148=0;|var149=0;|var150=0;|var151=0;|var152=0;|var153=0;|var154=0;|var155=0;|var156=0;|var157=0;|var158=0;|var159=0;|var160=0;|var161=0;|var162=0;|var163=0;|var164=0;|var165=0;|var166=0;|var167=0;|var168=0;|var169=0;|var170=0;|var171=0;|var172=0;|var173=0;|var174=0;|var175=0;|var176=0;|var177=0;|var178=0;|var179=0;|var180=0;|var181=0;|var182=0;|var183=0;|var184=0;|var185=0;|var186=0;|var187=0;|var188=0;|var189=0;|var190=0;|var191=0;|var192=0;|var193=0;|var194=0;|var195=0;|var196=0;|var197=0;|var198=0;|var199=0;|var200=0;|var201=0;|var202=0;|var203=0;|var204=0;|var205=0;|var206=0;|var207=0;|var208=0;|var209=0;|var210=0;|var211=0;|var212=0;|var213=0;|var214=0;|var215=0;|var216=0;|var217=0;|var218=0;|var219=0;|var220=0;|var221=0;|var222=0;|var223=0;|var224=0;|var225=0;|var226=0;|var227=0;|var228=0;|var229=0;|var230=0;|var231=0;|var232=0;|var233=0;|var234=0;|var235=0;|var236=0;|var237=0;|var238=0;|var239=0;|var240=0;|var241=0;|var242=0;|var243=0;|var244=0;|var245=0;|var246=0;|var247=0;|var248=0;|var249=0;|var250=0;|var251=0;|var252=0;|var253=0;|var254=0;|var255=0;|var256=0;|var257=0;|var258=0;|var259=0;|var260=0;|var261=0;|var262=0;|var263=0;|var264=0;|var265=0;|var266=0;|var267=0;|var268=0;|var269=0;|var270=0;|var271=0;|var272=0;|var273=0;|var274=0;|var275=0;|var276=0;|var277=0;|var278=0;|var279=0;|var280=0;|var281=0;|var282=0;|var283=0;|var284=0;|var285=0;|var286=0;|var287=0;|var288=0;|var289=0;|var290=0;|var291=0;|var292=0;|var293=0;|var294=0;|var295=0;|var296=0;|var297=0;|var298=0;|var299=0;|var300=0;|var301=0;|var302=0;|var303=0;|var304=0;|var305=0;|var306=0;|var307=0;|var308=0;|var309=0;|var310=0;|var311=0;|var312=0;|var313=0;|var314=0;|var315=0;|var316=0;|var317=0;|var318=0;|var319=0;|var320=0;|var321=0;|var322=0;|var323=0;|var324=0;|var325=0;|var326=0;|var327=0;|var328=0;|var329=0;|var330=0;|var331=0;|var332=0;|var333=0;|var334=0;|var335=0;|var336=0;|var337=0;|var338=0;|var339=0;|var340=0;|var341=0;|var342=0;|var343=0;|var344=0;|var345=0;|var346=0;|var347=0;|var348=0;|var349=0;|var350=0;|var351=0;|var352=0;|var353=0;|var354=0;|var355=0;|var356=0;|var357=0;|var358=0;|var359=0;|var360=0;|var361=0;|var362=0;|var363=0;|var364=0;|var365=0;|var366=0;|var367=0;|var368=0;|var369=0;|var370=0;|var371=0;|var372=0;|var373=0;|var374=0;|var375=0;|var376=0;|var377=0;|var378=0;|var379=0;|var380=0;|var381=0;|var382=0;|var383=0;|var384=0;|var385=0;|var386=0;|var387=0;|var388=0;|var389=0;|var390=0;|var391=0;|var392=0;|var393=0;|var394=0;|var395=0;|var396=0;|var397=0;|var398=0;|var399=0;|var400=0;|var401=0;|var402=0;|var403=0;|var404=0;|var405=0;|var406=0;|var407=0;|var408=0;|var409=0;|var410=0;|var411=0;|var412=0;|var413=0;|var414=0;|var415=0;|var416=0;|var417=0;|var418=0;|var419=0;|var420=0;|var421=0;|var422=0;|var423=0;|var424=0;|var425=0;|var426=0;|var427=0;|var428=0;|var429=0;|var430=0;|var431=0;|var432=0;|var433=0;|var434=0;|var435=0;|var436=0;|var437=0;|var438=0;|var439=0;|var440=0;|var441=0;|var442=0;|var443=0;|var444=0;|var445=0;|var446=0;|var447=0;|var448=0;|var449=0;|var450=0;|var451=0;|var452=0;|var453=0;|var454=0;|var455=0;|var456=0;|var457=0;|var458=0;|var459=0;|var460=0;|var461=0;|var462=0;|var463=0;|var464=0;|var465=0;|var466=0;|var467=0;|var468=0;|var469=0;|var470=0;|var471=0;|var472=0;|var473=0;|var474=0;|var475=0;|var476=0;|var477=0;|var478=0;|var479=0;|var480=0;|var481=0;|var482=0;|var483=0;|var484=0;|var485=0;|var486=0;|var487=0;|var488=0;|var489=0;|var490=0;|var491=0;|var492=0;|var493=0;|var494=0;|var495=0;|var496=0;|var497=0;|var498=0;|var499=0;|var500=0;|var501=0;|var502=0;|var503=0;|var504=0;|var505=0;|var506=0;|var507=0;|var508=0;|var509=0;|var510=0;|var511=0;|var512=0;|var513=0;|var514=0;|var515=0;|var516=0;|var517=0;|var518=0;|var5
```

[illegible]

Local control									
<pre>void Test1() {variousItems[1] = count+5; theCount;} void Test3() {variousItems[2] = localControlled[count+3]; theCount}</pre>									
Item	Value	Controlled	Controlled by	Controlled by	Controlled by	Controlled by	Controlled by	Controlled by	Controlled by
variousItems[0]	0	no							
variousItems[1]	5	yes	Test1	Test1	Test1	Test1	Test1	Test1	Test1
variousItems[2]	3	yes	Test3	Test3	Test3	Test3	Test3	Test3	Test3
variousItems[3]	0	no							
variousItems[4]	0	no							
variousItems[5]	0	no							
variousItems[6]	0	no							
variousItems[7]	0	no							
variousItems[8]	0	no							
variousItems[9]	0	no							
variousItems[10]	0	no							
variousItems[11]	0	no							
variousItems[12]	0	no							
variousItems[13]	0	no							
variousItems[14]	0	no							
variousItems[15]	0	no							
variousItems[16]	0	no							
variousItems[17]	0	no							
variousItems[18]	0	no							
variousItems[19]	0	no							
variousItems[20]	0	no							
variousItems[21]	0	no							
variousItems[22]	0	no							
variousItems[23]	0	no							
variousItems[24]	0	no							
variousItems[25]	0	no							
variousItems[26]	0	no							
variousItems[27]	0	no							
variousItems[28]	0	no							
variousItems[29]	0	no							
variousItems[30]	0	no							
variousItems[31]	0	no							
variousItems[32]	0	no							
variousItems[33]	0	no							
variousItems[34]	0	no							
variousItems[35]	0	no							
variousItems[36]	0	no							
variousItems[37]	0	no							
variousItems[38]	0	no							
variousItems[39]	0	no							
variousItems[40]	0	no							
variousItems[41]	0	no							
variousItems[42]	0	no							
variousItems[43]	0	no							
variousItems[44]	0	no							
variousItems[45]	0	no							
variousItems[46]	0	no							
variousItems[47]	0	no							
variousItems[48]	0	no							
variousItems[49]	0	no							
variousItems[50]	0	no							
variousItems[51]	0	no							
variousItems[52]	0	no							
variousItems[53]	0	no							
variousItems[54]	0	no							
variousItems[55]	0	no							
variousItems[56]	0	no							
variousItems[57]	0	no							
variousItems[58]	0	no							
variousItems[59]	0	no							
variousItems[60]	0	no							
variousItems[61]	0	no							
variousItems[62]	0	no							
variousItems[63]	0	no							
variousItems[64]	0	no							
variousItems[6									

LUAMETAFUN
the new interfaces
context 2021 meeting

Three subsystems

- The core of the engine is still `Tg1`. It stores there:
 - The `TK` instances are opened up by `La`. We can read out `La` from `Tg1` and to some extent from `La` to `Tg1`. Quite often we push back something to the `major` channels.
 - The `MetaMajor` file is accessed from `La`. Its contents can be read out. There is always `La` in between. Besides go back via `La`. The library can also read out `La` and from there to `Tg1`.
- This means that all three major components of `La` have `Tg1` can talk to each other and can catch other opportunities.
- With `La` in the center, we have access to internal operations, for instance faces, graphics and rendering.
- On the `Tg1` the `La` language also permits writing out, such as, `ng` databases and other types that can be dealt with programmatically.
- All has been reasonably optimized for efficiency and performance.

Insurrection - context 2011 meeting - three subsystems

[illegible]

- We don't need the Postscript backend (which only does TeX) anyway.
- We also have no way to get any input.
- The `lua` loader module has an advantage over the default one: it brings quite some `lua` standard library code.
- The `lua` loader module is not used in `luaTeX` because we handle it differently.
- All this means that we can do with a `small` (unmodified) `luaTeX` library.
- The conclusion has been drawn: we still have a `lua` thread to use as a `lua` script to convert the `TeX` which is not used, but we have control over how it comes out.
- A `luaTeX` library is this, managing handling etc, only largely goes to it. It is made for `lua`.
- The same is true for `lua` functions and return values: they can be adapted for `lua`.
- Memory management allocation is under engine control (we use `TeX` and `lua`), we use a custom high performance allocator.

MATH
complete control
context 2021 meeting

The benchmark

- Traditional "gc" has set the benchmark for typesetting math.
- Some other competitors of some, apfloat, apfloat, etc.
- Advantages in some lead to some benefits of the benchmark.
- The last family makes (some) changes to the second pass in one has to use very controlled fast switching.
- Some features (limits, integrals, printed only on special macros and parsing).
- One can end up in tricky fast switching. Also, some more features are loaded in order to set up the machinery.

math – current 2021 meeting – the benchmark

Today's reality

- We were not `Opport` type match items, in fact, in `NG47` we always used only one base instance (unless we were base `Instance`)
- We were `Opport` type `Opport` elements, and in the `NG47` there was no need to handle the number of `Opport` elements in `Opport`
- In `NG47` we used only a few base `Instance` regular: `ng2`, `bold`, `bold`, `ng2`
- All the `Opport` stuff is done with the help of `Opport`. This already started very early in the `NG47` project and has been going on for a long time
- We did not start into `Instance` because the available `Opport` are inconsistent, incompatible, have a lot of bugs and are changing
- We have a `Opport` type of `Opport` to place that was during the transition between the base and `Opport` was available
- In `NG47` there are various ways to deal with shortcomings in an extension to `Opport`
- At some point we need to make up our mind, accept the issues with `Opport`, and just fix things

— [math](#) — [control](#) [2023](#) [meeting](#) — [index](#) [reality](#)

The engine

- In `LaTeX` we started with a hybrid approach but eventually ended up with more rigid code paths.
- In `LaTeXmkIV` nearly all aspects of the engine had made configurable and are under user control.
- This permits experiments where we can apply old methods onto new fonts.

math — **summer 2023 meeting** — the engine

context-2021-overloadprotection

OVERLOAD PROTECTION

the downside of macros

context 2021 meeting

Primitives

A `\TgT` engine comes with a whole set of primitive operations for:

- accessing internal variables
- defining macros
- accessing expansion
- constructing boxes
- building pages
- defining characters (text and math)
- inserting boxes, glue and penalties
- defining fonts
- dealing with language (implementation)
- testing properties
- manipulating tables
- managing macros
- handling marks
- grouping
- subexpressions
- tracing

overload protection — context 2021 meeting — primitives

Macros

- Macros are commands defined by the user under a macro package.
- They can overload a primitive which then can confuse the whole machinery.
- A macro package can also primitives for instance `\relax` can be replaced by `\font\relax after` `\let\let\after\relax`.
- The only when for definition time the `_` as a letter. By using such a character some primitive against overload is provided.
- In ConTeXt we often, usually abuse like `\internal\let` but of course these can also be overloaded.
- We only overload a very few primitives, for instance `\language`.
- Users who overload primitives are 'on their own' and 'without support'.
- An easy way to protect yourself is using `\ContextCase macros`.

overload protection — context 2021 meeting — macros

Overload protection

- The `\LoadTgT` engine has overload protection built in for the `\TgT` engine as well as provides means to do that for `\MetaPost`.
- In L^AT_EX all commands have been tagged accordingly (which was quite some work).
- Processing a system macro will then do no harm.
- Overload protection is off by default but can be turned on:

```
\setobj{objectives}{overload=warning}
\setobj{objectives}{overload=warn}
```
- `\unpaved` always run with the error raised and make sure that the macros obey the rules.
- Modules and/or styles (and in a few cases the more could run cheap and use `\unpaved\loadmodule`):

```
.....
\unpaved\loadmodule
```

overload protection — context 2021 meeting — overload protection

context-2021-paragraphs

PARAGRAPHS

a bit of an upgrade

context 2021 meeting

Note

- Some of the following already is present for a while and has been discussed at previous meet-ups.
- But... occasionally some minor tweaks you added as consider this to be an update.

paragraphs — context 2021 meeting — note

Spacing

- Spaces in `\TgT` because glue nodes (with optional stretch and shrink).
- In traditional `\TgT` these glue nodes are not extended copies of the current spacing related values.
- In ConTeXt we make real copies so that when we mess with the node list changes to glue don't affect other instances.

paragraphs — context 2021 meeting — spacing

Parameters

- In traditional `\TgT` the paragraphs based properties that are in effect when `\par` happens are used when breaking sections.
- In `\LoadTgT` the paragraphs based properties are stored with the paragraph and can be frozen when they are set.
- This gives a more predictable (and robust) way of manipulating a paragraph.
- We can for instance get rid of grouping side effects that interfere with `\overprint`.
- Currently three dozen parameters are tracked but they are grouped in categories.
(show code and examples)

paragraphs — context 2021 meeting — parameters

context-2021-programming

PROGRAMMING

the way ConTeXt is set up

context 2021 meeting

Levels

When you look at ConTeXt bottom-up (engine—interface) you will notice:

1. **primitives**: that is what the engine comes with
2. **infrastructure**: basic management of data structures
3. **helpers**: macros that hide complexity
4. **subsystems**: reflections of macros that implement functionality
5. **mechanisms**: these combine various subsystems
6. **modules**: extra functionality (used 1-5)
7. **styles**: handling sources and layout issues 4-6)

Users normally use ConTeXt top-down (usage—backlog).

programming — context 2021 meeting — levels

Styles

- These are probably solutions for common as well as rare situations.
- The system comes with some styles: the `x-y` files.
- Right from the start the idea was that you get some reasonable default.
- And if you want more you yourself define your style as you go.
- It is really a part of the game: exploration.
- Solving your problem is a nice challenge.
- If you want a completely predefined setup, drop somewhere else.

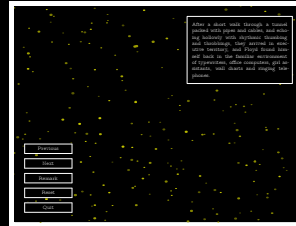
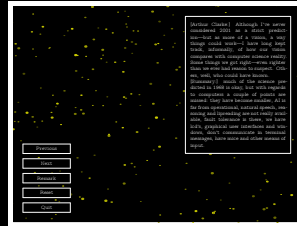
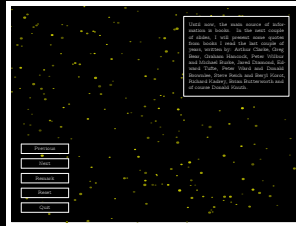
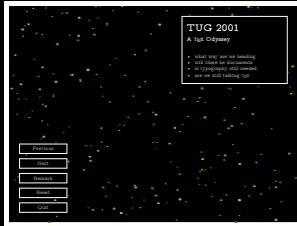
programming — context 2021 meeting — styles

Modules

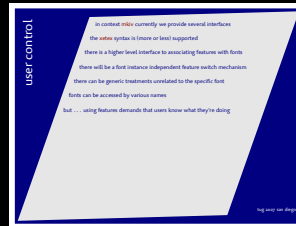
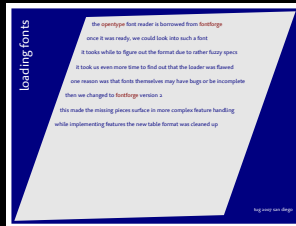
- A lot of functionality is built in.
- This helps to keep the system consistent.
- We could cheat and skip thousands of low-level styles but don't do that.
- There are a few mechanisms that don't really fit into the core, so these are implemented as modules that do fit into the interface: `foo-x` and `x-foo`.
- Users can build and share their solutions which has resulted in some third party modules: the `x-y` files.
- For a few, when old people like `foo-x` > `name` schemes.

programming — context 2021 meeting — modules

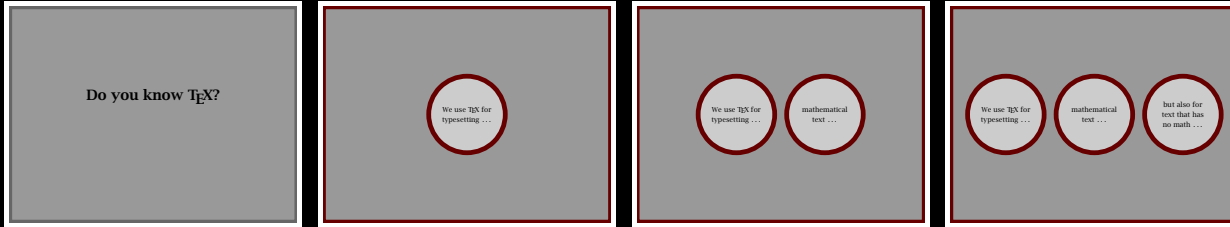
tug-2001-ideas



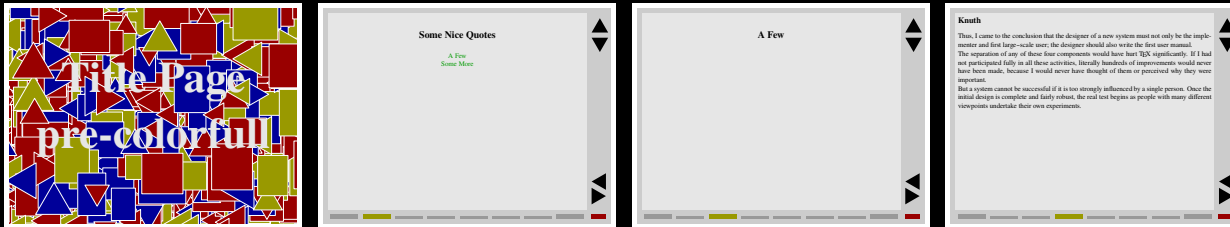
tug-2007-fonts



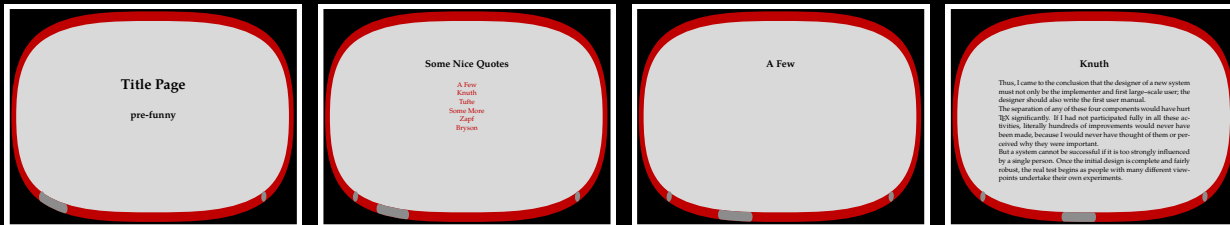
present-balls-001



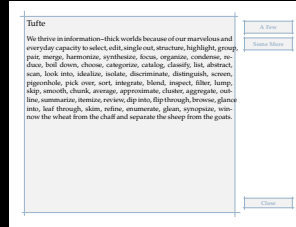
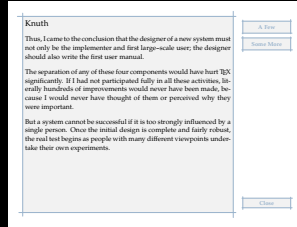
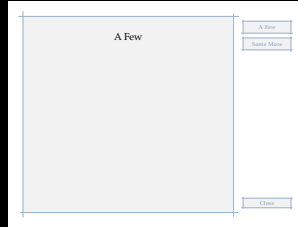
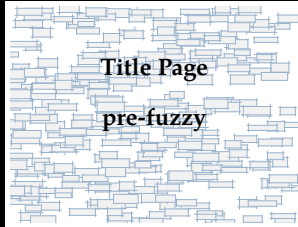
present-colorful-001



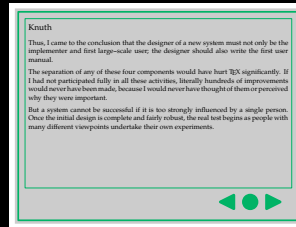
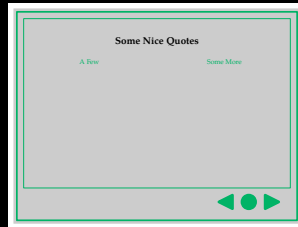
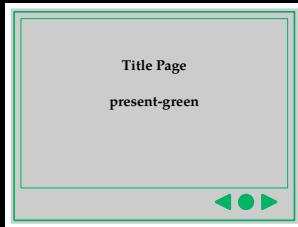
present-funny-001



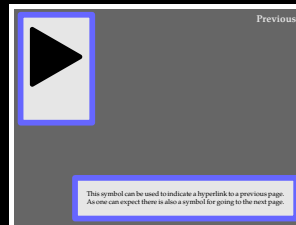
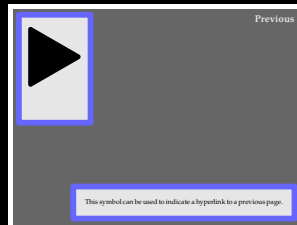
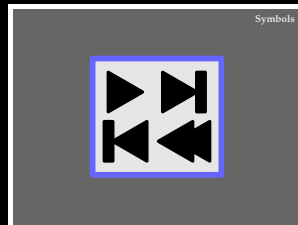
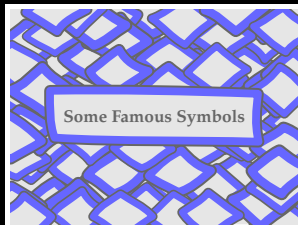
present-fuzzy-001



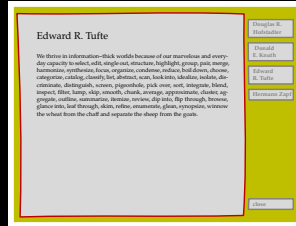
present-green-001



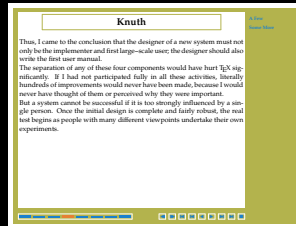
present-grow-001



present-original-001



present-original-001

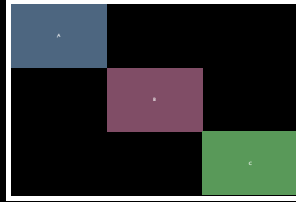
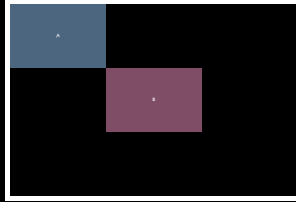


present-punk-001

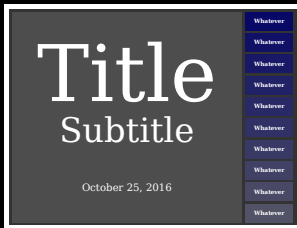


Examples

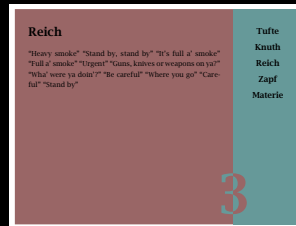
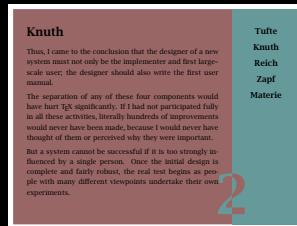
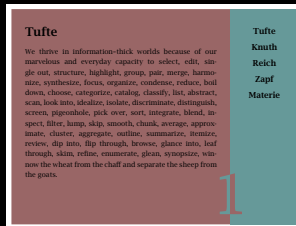
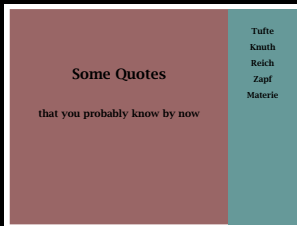
present-random-001



present-shaded-001



present-split-001



present-stepper-001

Stepwise
Refinement

Topics

Female Artists
Male Composers

And Some More

Female Artists

- Fiona Apple
- Tori Amos
- Kate Bush
- Heather Nova
- Alanis Morissette
- Suzanne Vega

Male Composers

- John Adams
- Steve Reich
- Louis Andriessen
- Olivier Messiaen

present-steps-001

Contents

1 Step Set 1 2
2 Step Set 2 1

1 Step Set 1

1 Step Set 1

STEP ONE

1 Step Set 1

STEP ONE

STEP TWO

present-tiles-001

Whatever We
Want Here

Whatever We
Want There

Topic 1	Topic 2	Topic 3
Topic 4	Topic 5	Topic 6
Topic 7	Topic 8	Topic 9
Topic 10	Topic 11	Topic 12

Topic 1

We thrive in information-thick worlds because of our marvelous and everyday capacity to select, edit, single out, structure, highlight, group, pair, merge, harmonize, synthesize, focus, organize, condense, reduce, boil down, choose, categorize, catalog, classify, list, abstract, scan, look into, identify, isolate, discriminate, distinguish, screen, pigeonhole, pick over, sort, integrate, blend, inspect, filter, bury, skip, smooth, check, average, approximate, cluster, aggregate, outline, summarize, itemize, review, dip into, flip through, browse, glance into, lead through, skim, refine, enumerate, plan, reorganize, rewire the subject from the chaff and separate the sheep from the goats.

Buttons (roll-over in acrobath):

click left top home
click right top contents
click left bottom previous
click left bottom next page

Topic 2

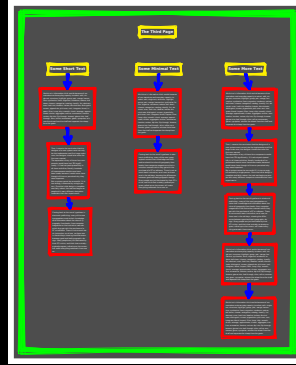
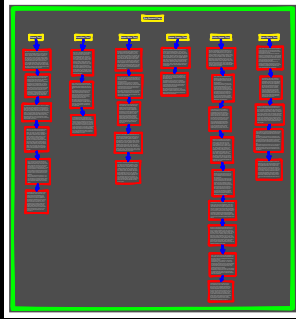
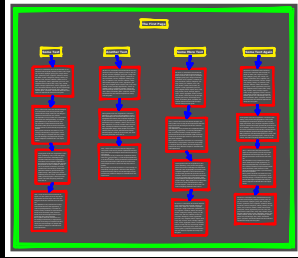
We thrive in information-thick worlds because of our marvelous and everyday capacity to select, edit, single out, structure, highlight, group, pair, merge, harmonize, synthesize, focus, organize, condense, reduce, boil down, choose, categorize, catalog, classify, list, abstract, scan, look into, identify, isolate, discriminate, distinguish, screen, pigeonhole, pick over, sort, integrate, blend, inspect, filter, bury, skip, smooth, check, average, approximate, cluster, aggregate, outline, summarize, itemize, review, dip into, flip through, browse, glance into, lead through, skim, refine, enumerate, plan, reorganize, rewire the subject from the chaff and separate the sheep from the goats.

Buttons (roll-over in acrobath):

click left top home
click right top contents
click left bottom previous
click left bottom next page

Examples

present-weird-001



present-windows-001

